

IWI131 Programación (Fase 2) — Ejercicios Semana 3: Diccionarios y Archivos

1. *Python airlines* mantiene un exhaustivo recuento de todos sus vuelos y pilotos. Cada uno de los vuelos tiene un identificador único y al igual que cada piloto. La empresa ha almacenado en el archivo `itinerario2018.txt` todos los vuelos realizados el año anterior, donde cada línea tiene el formato `id_vuelo:origen/destino/kilometros`:

`itinerario2018.txt`

```
NAC23:Santiago/Iquique/1600
INT12:Sao Paulo/Paris/9401
NAC22:Lyon/Paris/400
INT33:Bogota/Santiago/6383
NAC49:Bogota/San Andres/1209
NAC34:Iquique/Punta Arenas/3600
NAC25:Punta Arenas/Santiago/2000
...
```

Además se tiene un segundo archivo con todos los vuelos realizados por cada piloto, llamado `VP2018.txt` donde cada línea tiene el formato `id_piloto-mes:id vuelo1;id vuelo2,...`:

`VP2018.txt`

```
12-marzo:INT12;NAC22
11-marzo:NAC23;INT33;NAC49
3-noviembre:NAC34;NAC25
...
```

Se pide que desarrolle la función `resumen_piloto(mes)` que reciba el mes para el cual se quiere realizar un resumen. La función debe escribir un archivo `resumen_X.txt` donde *X* es el nombre del mes y en cada línea del archivo debe contener el `id_piloto` y la cantidad de **kilómetros totales** recorridos en sus vuelos, separado por un `'&'`.

Solo deben aparecer los pilotos que tengan viajes en ese mes, si no existe ningún viaje durante el mes pedido debe escribir un mensaje en el archivo indicando la situación. Por ejemplo, al llamar la función con el mes "marzo": `resumen_piloto("marzo")` se creará el archivo `resumen_marzo.txt` mostrado a continuación.

`resumen_marzo.txt`

```
12&9801
11&9192
```

2. Es fin de año y la revista Pylboard debe publicar dos rankings en función de los videos que están en la plataforma PyTube: el grupo de las 10 canciones con más visitas y los 10 artistas con mayor cantidad de visitas sumando todas sus canciones. Para ello escribieron en un archivo todos los artistas a considerar. Cada línea contiene un artista diferente.

Por otro lado, cada artista tiene su propio archivo de texto donde por cada línea está el título de una canción, un espacio en blanco y su respectiva cantidad de visitas, pero donde hay un punto cada 3 dígitos (de derecha a izquierda) para facilitar la lectura. Notar además que el nombre del archivo de texto de un artista es su nombre en minúsculas, sin considerar espacios en blanco, más la extensión “.txt”. A continuación se muestran ejemplos de los archivos (notar que no hay orden establecido para los datos):

artistas.txt

```
Pylor Pyft
Juan Pys Guerra
CalPyn Harris
Dua Pypa
...
```

calpynharris.txt

```
How Py is your love 89.000.331
This Is What You Py For 22.298.098
Pystroke 789.322
One Py 78.983.029
...
```

La revista lo contrata a usted para que elabore la función `rankings(arch_art)`, donde `arch_art` es un string que corresponde al nombre del archivo donde están todos los artistas (como “artistas.txt”). La función debe escribir los archivos de los dos ranking solicitados, donde por cada línea estará la posición (en orden ascendente), el nombre de la canción o artista y el número de visitas, separados por espacios en blanco. Suponiendo que “Pylor Pyft” es la artista con más visitas y la canción “How Py is your love” (que no necesariamente corresponde a “Pylor Pyft”) es la que tiene más visitas, al ejecutar el programa se deberían escribir los siguientes archivos y retornar la tupla (“How Py is your love”, “Pylor Pyft”).

rank_canciones.txt

```
1 How Py is your love 89000331
2 Look what you Py do 88999002
...
10 PyLove 991020
```

rank_artistas.txt

```
1 Pylor Pyft 533994012
2 CalPyn Harris 191070780
...
10 Justin Pyber 8967
```

Notar que al escribir los números en estos archivos no se ponen puntos como separador.

El presidente de Pylboard le indica además que no se preocupe por eventuales empates, simplemente debe ordenar según mayor cantidad de visitas y considerar hasta el décimo, sabiendo que siempre habrán muchos más. Recuerde que pueden haber miles de artistas y canciones.

3. *PythExpress* es la empresa de transporte internacional más importante de Pythonia, y ofrece el servicio de traslado de mercancías desde el exterior, encargándose de los trámites de importación y pago de impuestos. Los envíos que se encuentran en trámite se almacenan en un archivo como el que se muestra a continuación, en el que cada línea corresponde a un paquete enviado:

paquetes.txt

```
11111111-1:50:SI:Juguetes:0.5:2.5
11111111-1:150:SI:Telefono:5:6.2
22222222-2:40:NO:Libros:0.08:1.5
33333333-3:25:SI:Reloj:0.13:2.2
```

El formato de cada línea es RUT:ValorOriginal:PagaArancel:descripción:peso_volumetrico:peso. El ValorOriginal corresponde al valor del producto en dólares (US\$). El campo PagaArancel puede ser únicamente SI o NO, y se usa para saber si la mercancía enviada debe pagar impuestos. Algunos bienes no pagan este arancel, como por ejemplo los libros.

PythExpress calcula el costo de transporte de la siguiente manera. Primero, determina el mayor valor entre el peso del paquete y el peso volumétrico. El mayor de estos valores se multiplica por la tarifa establecida de US\$2 por Kg.

El costo final a pagar por el usuario es el valor original más el costo de transporte. Si el paquete paga arancel, a este costo final se le debe sumar el 19 %.

La empresa mantiene el registro de sus clientes en un archivo como el siguiente:

clientes.txt

```
11111111-1;Andrea;Viña del Mar
22222222-2;Raquel;Valparaíso
33333333-3;Miguel;Quillota
```

Escriba la función `crear_archivos(nombre_archivo_envíos, nombre_archivo_clientes)` que recibe como parámetros los nombres de los archivos con la información descrita. La función debe crear un archivo para cada cliente que haya recibido envíos, con el nombre `importaciones-NOMBRE.txt`, donde NOMBRE corresponde al nombre del cliente. Puede suponer que todos los clientes que aparecen en `nombre_archivo_clientes` tienen al menos un pedido registrado en `nombre_archivo_envíos`. Guíese por el ejemplo.

Ejemplo:

```
>>> print(crear_archivos('paquetes.txt', 'clientes.txt'))
```

importaciones-Andrea.txt

```
Juguetes
Valor:50 - Final:65.45

Telefono
Valor:150 - Final:193.256

Total:258.706
```

4. Una empresa de productos ha comenzado a vender sus artículos fuera de Chile. Cada producto que será enviado, cuenta con un código único para poder ser identificado. Para que un producto llegue a destino, muchas veces debe realizar un viaje por distintos países antes de llegar al cliente. Esta información es guardada en el archivo *envios.txt*, donde cada línea tiene el formato *codigo:paisOrigen-pais2-...paisDestino*, donde siempre el primer país es el de origen y el último el de destino. Los países intermedios son aquellos por los cuales debe pasar para llegar a destino, notar que pueden existir más de un país intermedio.

envios.txt

```
RN1021:chile-peru-colombia-panama
NB812:chile-peru-ecuador-brasil
KM192:chile-peru-bolivia-brasil
UY291:chile-peru-australia
LA190:chile-argentina-uruguay-brasil
IW131:chile-panama
...
```

tiempos.txt

```
chile+peru=2$70000
colombia+peru=1$40000
chile+argentina=2$65000
peru+bolivia=2$30000
bolivia+brasil=3$50000
argentina+uruguay=1$25000
peru+ecuador=3$55900
uruguay+brasil=3$75000
peru+australia=5$103500
panama+chile=4$95000
panama+colombia=1$58400
ecuador+brasil=2$41000
...
```

Además, durante el tiempo se ha logrado conocer los días que demora cada tramo y el costo monetario. Para ello, se tiene el archivo *tiempos.txt*, donde cada línea tiene el formato *pais1+pais2=dias\$costo*. Considere que el costo de ir de pais1 a pais2 es el mismo en ambos sentidos. Asuma que en este archivo se encuentran todos los tramos posibles, pero se presentan en un sólo sentido, es decir, si está chile y peru no existe la línea peru+chile=2\$70000, en el archivo tiempos.txt.

Desarrolle la función **generar_reporte(maxd)** que recibe la máxima cantidad de días que se puede demorar un viaje desde el origen a destino considerando todos los tramos que debe realizar. La función debe crear un archivo llamado *X_dias.txt*, donde X es la cantidad de días recibidos como parámetro. Este archivo debe guardar en cada línea el código del producto y los días que tardará en llegar en el formato *codigo:dias*.

Los viajes deben estar ordenados por el valor del envío. De más caro a más barato, aunque esta información no se debe escribir en el archivo. En caso de no existir viajes que cumplan con la condición, en el archivo se debe escribir el mensaje *No Hay viajes para esta cantidad de dias*

6_dias.txt

```
RN1021:4
LA190:6
IW131:4
```

3_dias.txt

```
No hay viajes con esta cantidad de dias
```

5. Los tomodatchi son mascotas virtuales que deben ser alimentadas constantemente o sino pueden morir. Como ejemplo, suponga que tiene la siguiente colección de tomodatchis en un archivo llamado `tomodatchi.txt` y los alimentos que pueden comer en el archivo `comidas.txt`. Notar que los dos números contiguos a un tomodatchi o alimento corresponden a la felicidad y satisfacción respectivamente:

`tomodatchi.txt`

```
pedrotchi;5;5
javiertchi;0;3
nicolatchi;3;0
cristophertchi;4;4
viktortchi;2;1
...
```

`comidas.txt`

```
sopaipa;1;2
tocomple;2;1
susushi;1;1
supapajohns;3;3
sumaruchan;1;0
...
```

Nota: Los archivos de ejemplo tienen más líneas.

a.- Desarrolle la función `status(tomo, arch)`, donde `tomo` es un string con el nombre del tomodatchi y `arch` un string con el nombre del archivo de tomodatchis. La función retorna un diccionario con la felicidad y satisfacción de dicho tomodatchi. Si la felicidad o la satisfacción es cero, entonces retorna `False` (tomodatchi muerto). Asuma que el tomodatchi siempre existirá en el archivo. Guíese por el ejemplo.

```
>>status('pedrotchi','tomodatchi.txt')
{'feliz': 5, 'satisfecho': 5}
>>status('javiertchi','tomodatchi.txt')
False
```

b.- Desarrolle la función `alimento(ali, arch)` donde `ali` es un string con el nombre del alimento y `arch` un string con el nombre del archivo de alimentos. La función retorna un diccionario con la felicidad y satisfacción que produce dicho alimento. Si no lo encuentra retorna `False`. Guíese por el ejemplo.

```
>>alimento('tocomple','comidas.txt')
{'feliz': 2, 'satisfecho': 1}
>>alimento('salchipapa','comidas.txt')
False
```

c.- Desarrolle la función `alimentar(tomo, ali, arch1, arch2)`, donde `tomo` es el nombre del tomodatchi, `ali` es el nombre del alimento, `arch1` es el nombre del archivo con los tomodatchis y `arch2` el nombre del archivo con los alimentos. En el caso de que el tomodatchi esté vivo y que el alimento exista, la función retorna un diccionario con los datos actualizados (sumándole la cantidad de satisfacción y felicidad correspondiente al alimento). Si el tomodatchi está muerto, la función retorna un string `'X.X'`, exista o no el alimento. Finalmente, si el tomodatchi está vivo y el alimento no existe, la función retorna un string `'no existe alimento'`. Asuma que el tomodatchi siempre existirá en el archivo. Guíese por el ejemplo:

```
>>alimentar('pedrotchi','tocomple','tomodatchi.txt','comidas.txt')
{'feliz': 7, 'satisfecho': 6}
>>alimentar('javiertchi','tocomple','tomodatchi.txt','comidas.txt')
X.X
>>alimentar('viktortchi','salchipapa','tomodatchi.txt','comidas.txt')
no existe alimento
```
